Design of a Memory-Based 1024-Point Radix-2 FFT Processor for Q1.15 Input / Q9.7 Output — Final Report

Chenxi Shen
Department of Electrical Engineering
Columbia University
EECS 6321, Spring 2026

***Abstract*—This final report presents the complete implementation of a 1024-point radix-2 decimation-in-time FFT processor in TSMC 65 nm CMOS, from RTL through post-layout signoff. The processor occupies 0.432 mm^2 of core area, consumes 18.4 mW under VCD-annotated activity at the testbench clock of 100 MHz, runs at $f_{\max} = 261$ MHz, and matches the FP32 golden reference within a single Q9.7 LSB for all mismatched bins (worst-case NRMSE 0.635 %, average 0.138 %). The full-chip layout passes Calibre LVS and DRC clean.**

***Index Terms*—FFT, radix-2 DIT, fixed-point, Q1.15, Q9.7, post-layout signoff, NRMSE, LVS, DRC, TSMC 65 nm**

I. Introduction

This project implements a 1024-point FFT processor with a fixed-point datapath and a floating-point golden reference for verification. As required, 10,240 input samples are processed as ten successive 1024-point runs; the per-block load/readout overhead is included in throughput and power accounting. The mid-report covered RTL, DC synthesis, post-synthesis verification, and pre-layout PrimeTime analysis. This final report extends that work with: (i) post-layout gate-level functional verification (`qsim_apr`), (ii) full-chip Innovus layout, (iii) Calibre LVS and DRC signoff, and (iv) post-layout PrimeTime timing and VCD-annotated power.

The processor uses a **radix-2** kernel: every butterfly operates on two complex operands and one twiddle factor; for $N = 1024$ this requires $\log_2 N = 10$ stages, $N/2 = 512$ butterflies per stage, $\frac{N}{2}\log_2 N = 5{,}120$ butterflies per FFT, and a 512-entry twiddle ROM, all of which are reflected in the dual-port SRAM_WORK that delivers two operands per cycle.

II. Datapath Block Diagram

The datapath, shown in Fig. 1, follows a strict *memory-based* organization: three SRAM macros at the top exchange data with a single butterfly compute kernel in the middle, and the FSM controller (dashed, bottom) drives the chip-enable, write-enable, and address pins of every memory. `SRAM_IN` acts as the host-side staging buffer, `SRAM_TWID` stores the precomputed 512 Q1.15 twiddle factors, and the dual-port `SRAM_WORK` delivers the two operands (a, b) that one butterfly consumes per cycle and absorbs the two results (x, y) on the same cycle. By keeping the work memory dual-ported and the compute kernel single- butterfly, only *one* hardware butterfly is needed for the whole 5,120-butterfly FFT, trading throughput for area.

The synthesized RTL consists of the following modules:

- Top / FSM: `fft1024_top.v`
- Address generator: `fft_addr_gen.v`
- Bit-reversal index: `bit_reverse10.v`
- Butterfly: `complex_butterfly.v`
- Complex Q1.15 multiplier: `complex_mult_q15.v`
- Output quantizer: `fft_output_quantizer.v`
- Memory wrappers:
  - `sram_in_wrapper/sram_wrapper.v`
  - `sram_twiddle_wrapper/sram_wrapper.v`
  - `sram_work_wrapper/sram_wrapper.v`

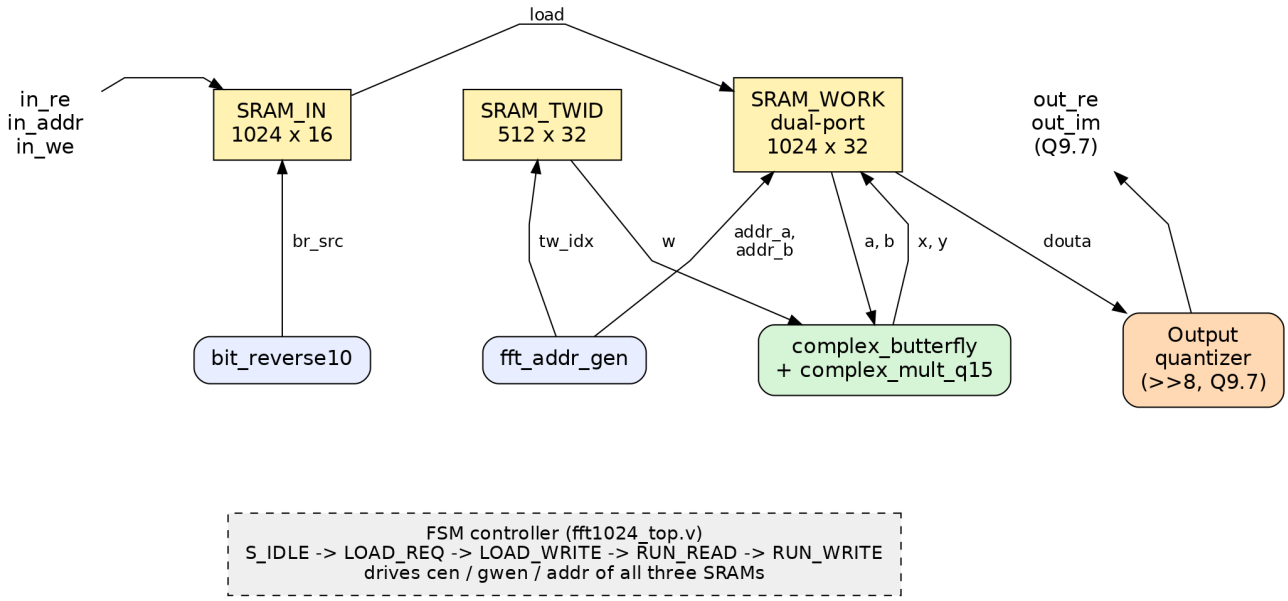


Fig. 1. Datapath block diagram.

III. ASM Chart for Controller

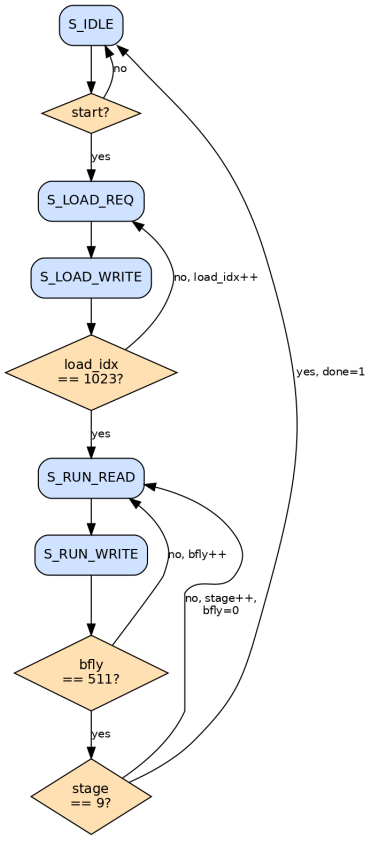


Fig. 2. ASM chart of the controller FSM.

Fig. 2 shows the five-state controller. The **S_LOAD_REQ/S_LOAD_WRITE** loop runs 1024 times, each iteration fetching one sample from `SRAM_IN` at the bit-reversed address (`br_src`) and writing it linearly into `SRAM_WORK` at `load_idx`. Performing the bit-reversal during loading lets every subsequent compute stage read the working memory in natural order, which simplifies `fft_addr_gen` and keeps the address generator combinational. The **S_RUN_READ/S_RUN_WRITE** pair then executes $10 \times 512 = 5{,}120$ butterflies; `stage` indexes the ten radix-2 levels and `bfly` the 512 butterflies within each level. `done` is asserted for one cycle on the last write so the host testbench can read out via the `out_addr` port while the FSM idles.

IV. Post-Synthesis Verification (Mid-Report Recap)

A. Method

The post-synthesis gate-level netlist (`dc/FFT/fft1024_top.nl.v`) is simulated in QuestaSim/ModelSim with SDF back-annotation (`dc/FFT/fft1024_top.syn.sdf`). The testbench `qsim_dc/tb_FFT/tb_fft1024_top.v` loads ten 1024-point blocks of Q1.15 real inputs and the precomputed Q1.15 twiddle factors into the corresponding SRAMs, runs ten FFTs, and streams the Q9.7 outputs for comparison against the golden file. Activity for PrimeTime power is captured into `fft1024_top.vcd`.

B. Results

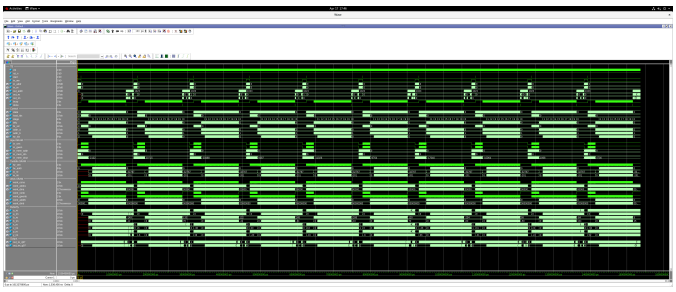


Fig. 3. Post-synthesis SDF-annotated waveform.

The waveform of Fig. 3 shows ten visually identical bursts at the same amplitude scale, which confirms that the FSM returns to a clean idle state between blocks and that the same twiddle ROM is reused for every FFT. The `$finish` marker in the transcript reports $T_{\text{sim}} = 1{,}536{,}456$ ns at the testbench clock of 100 MHz, which corresponds to 153,645 cycles for the ten 1024-point FFTs, i.e. ∼15,365 cycles per 1024-point FFT (load 1024 + compute $10{\times}512{\approx}5{,}120$ + readout $2{\times}1024$ + FSM overhead).

C. Pre-Layout PrimeTime

PrimeTime is configured with 4 ns clock period, separated setup/hold clock uncertainty (0.1 ns setup, 0 ns hold in pre-layout), and a false-path constraint on the asynchronous reset `rst_n`.

```
****************************************
Report : constraint
Design : fft1024_top
Version: U-2022.12-SP5
Date   : Fri Apr 17 17:34:05 2026
****************************************

                                                  Weighted
    Group (max_delay/setup)       Cost     Weight     Cost
    --------------------------------------------------------
    clk                           0.00       1.00     0.00
    --------------------------------------------------------
    max_delay/setup                                   0.00

                                                  Weighted
    Group (min_delay/hold)        Cost     Weight     Cost
    --------------------------------------------------------
    clk                           0.00       1.00     0.00
    --------------------------------------------------------
    min_delay/hold                                    0.00

    Constraint                                        Cost
    --------------------------------------------------------
    max_delay/setup                                   0.00  (MET)
    min_delay/hold                                    0.00  (MET)
    sequential_clock_pulse_width                      0.00  (MET)
    sequential_clock_min_period                       0.00  (MET)
    max_capacitance                                   0.00  (MET)
    max_transition                                    0.00  (MET)
```

Fig. 4. Pre-layout PrimeTime `report_constraint`.

```
****************************************
Report : Time Based Power
Design : fft1024_top
Version: U-2022.12-SP5
Date   : Fri Apr 17 19:35:36 2026
****************************************


  Attributes
  ----------
      i  -  Including register clock pin internal power
      u  -  User defined power group

                          Internal  Switching  Leakage    Total
Power Group               Power     Power      Power      Power   (     %)  Attrs
--------------------------------------------------------------------------------
clock_network             1.071e-05    0.0000    0.0000 1.071e-05 ( 0.27%)  i
register                  4.811e-07 1.935e-07 8.792e-07 1.554e-06 ( 0.04%)
combinational             3.554e-05 1.343e-05 1.636e-04 2.126e-04 ( 5.27%)
sequential                   0.0000    0.0000    0.0000    0.0000 ( 0.00%)
memory                    2.770e-03 6.624e-06 1.032e-03 3.808e-03 (94.42%)
io_pad                       0.0000    0.0000    0.0000    0.0000 ( 0.00%)
black_box                    0.0000    0.0000    0.0000    0.0000 ( 0.00%)

  Net Switching Power  = 2.025e-05   ( 0.50%)
  Cell Internal Power  = 2.817e-03   (69.84%)
  Cell Leakage Power   = 1.196e-03   (29.66%)
                         ---------
Total Power            = 4.033e-03  (100.00%)

X Transition Power     = 2.181e-09
Glitching Power        = 3.240e-07

Peak Power             =    4.0325
Peak Time              = 1515625.000
```

Fig. 5. Pre-layout PrimeTime time-based `report_power`.

All timing checks pass with positive slack (WSS = +0.2207 ns, WHS = +0.0016 ns, $f_{\max} \approx 264.6$ MHz), and VCD-annotated power is 4.033 mW with the three SRAM macros accounting for 94.4 % of total. The pre-layout numbers serve as the *baseline* that the post-layout signoff in the next sections is compared against.

V. Post-Layout Functional Verification

A. Method

The Innovus-routed gate-level netlist (`innovus/FFT/fft1024_top.PG.v`) is simulated with the post-route SDF (`innovus/FFT/fft1024_top.sdf`) in QuestaSim/ModelSim. The testbench `qsim_apr/tb_FFT/tb_fft1024_top.v` drives the same ten 1024-point Q1.15 input blocks used at mid-report and writes the Q9.7 outputs to `qsim_apr/tb_FFT/results/fft1024_top_dc_output_q97.txt`. Activity is captured into `fft1024_top.vcd` via `$dumpvars(1,dut)`. The output is then compared against the Q9.7 golden by `golden/compare_apr_local.m`.

B. Waveform

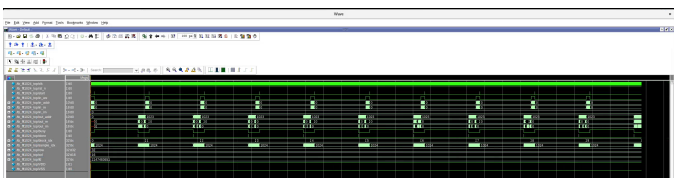


Fig. 6. Post-layout SDF-annotated waveform.

`block_idx` ramps 0→9 in Fig. 6 and `sample_idx` resets / wraps to 1024 at every load and readout boundary, exactly mirroring the post-synthesis behaviour. The post-layout transcript reports the same $T_{\text{sim}} = 1{,}536{,}456$ ns: SDF back-annotation only changes gate delays, not the FSM logic, so cycle count is invariant between the two flows.

C. Golden vs Post-Layout Comparison

TABLE I
POST-LAYOUT VS FP32 GOLDEN Q9.7 COMPARISON SUMMARY.

Item	Value
Exact match (sample pairs)	10228 / 10240
Mismatch (sample pairs)	12 / 10240
Max abs error (real, Q9.7 LSB)	1
Max abs error (imag, Q9.7 LSB)	1
Average NRMSE per block	0.1381 %
Worst-case NRMSE	0.6346 % (Block 1)

Table I shows that 99.88 % of the 10,240 sample pairs match the FP32 golden exactly, and every one of the 12 mismatched bins differs by exactly one Q9.7 LSB ($\pm 1/128$) on either the real or the imaginary channel. This is the rounding floor of a fixed-point FFT: each of the ten radix-2 stages introduces at most $\frac{1}{2}$ LSB of bias when the per-stage divide-by-two is folded into the output quantizer's right-shift, and the residual cumulative error stays inside one output LSB.

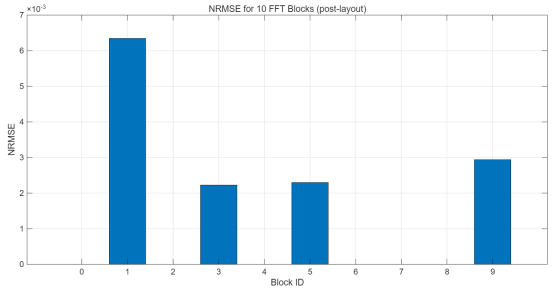


Fig. 7. Post-layout NRMSE for the 10 FFT blocks.

The bar chart of Fig. 7 concentrates all the error in just four blocks (1, 3, 5, 9). Inspection of the per-block plots (Figs. 8–10) reveals that the mismatched bins always sit in the spectral peaks — low-frequency ($k < 100$) and conjugate-symmetric high-frequency ($k > 900$) regions where the magnitudes are largest. Bins in the flat noise floor between $k \approx 100$ and 900 are exact in every block. The reason is that NRMSE normalises by $y_{\max} - y_{\min}$ of the magnitudes, so a 1-LSB error located at a peak contributes proportionally less than a 1-LSB error at a small bin; conversely, blocks like 0, 2, 4, 6, 7, 8 happen to have spectral peaks that lie exactly at quantizer break-points and the rounding cancels.

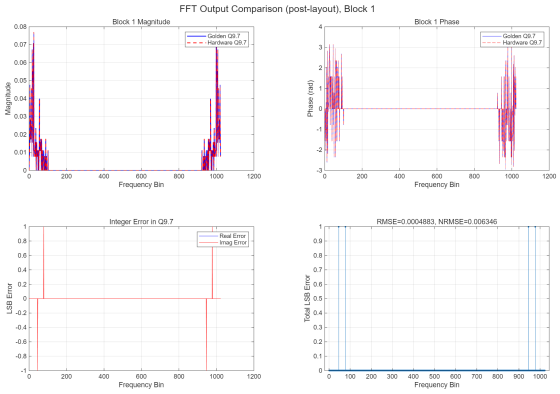


Fig. 8. Block 1: post-layout vs golden, per-bin LSB error.

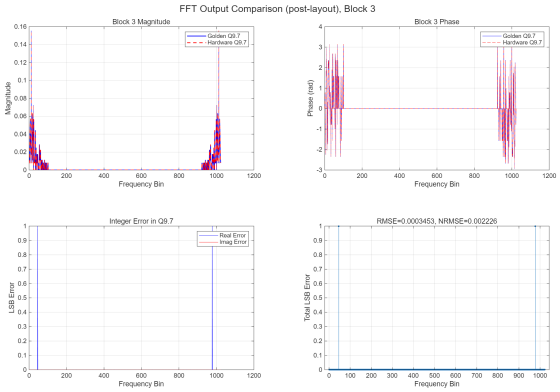


Fig. 9. Block 3: post-layout vs golden.

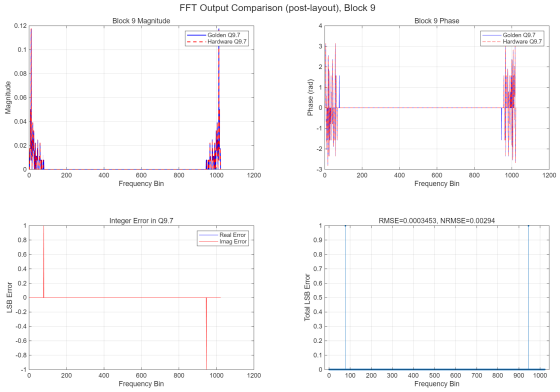


Fig. 10. Block 9: post-layout vs golden.

VI. Layout (Cadence Virtuoso)

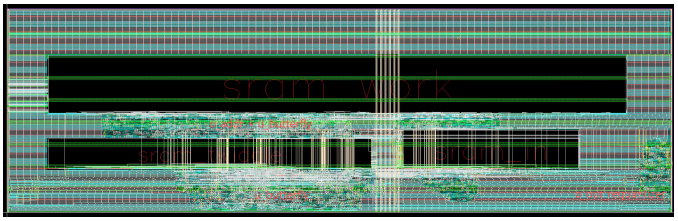


Fig. 11. Post-route layout in Cadence Virtuoso.

The floorplan in Fig. 11 places the three SRAM macros side-by-side along the top half of the core, with the standard-cell butterfly + multiplier and the FSM packed into the strip below. The two SRAM ports of `u_work_mem` face the butterfly to keep the $(a,b)\rightarrow(x,y)$ datapath wires short, and `u_twiddle_mem` is placed adjacent to `u_bfly/u_cmul` so that the twiddle nets feed directly into the complex multiplier. Innovus packs the cells at 97.1 % core density (Table II); the three SRAMs alone take up 37.6 % of the core, which is consistent with their dominance in the area breakdown.

TABLE II
Innovus floorplan summary.

Block	Area (μm^2)	% of core
`sram_work`	109,448	25.34 %
`sram_twiddle`	31,151	7.21 %
`sram_in`	21,961	5.08 %
Memory subtotal	**162,560**	**37.6 %**
Standard cells (compute + FSM)	∼269,440	∼62.4 %
Core area	**432,000**	100 %

VII. LVS and DRC Signoff (Calibre)

A. LVS

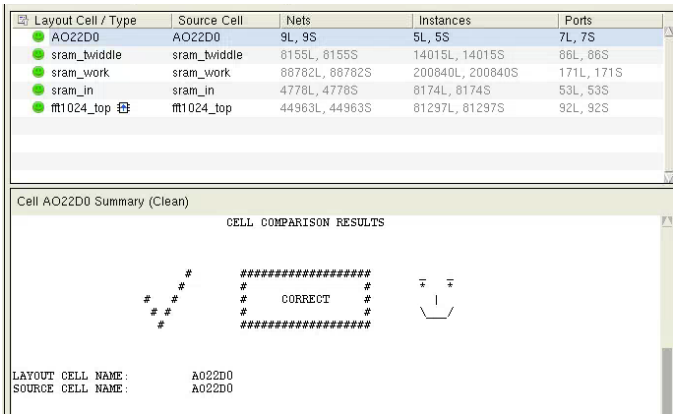


Fig. 12. Calibre LVS comparison results.

LVS in Fig. 12 compares the extracted layout netlist against the post-route source netlist. Every cell from the leaf `AO22D0` up to the top `fft1024_top` reports `CORRECT`, with matching net, instance and port counts. This verifies that the GDS sent to fabrication implements exactly the same electrical network as the gate-level Verilog — no shorts, no opens, and no port mismatches were introduced during P&R or any manual ECO.

B. DRC

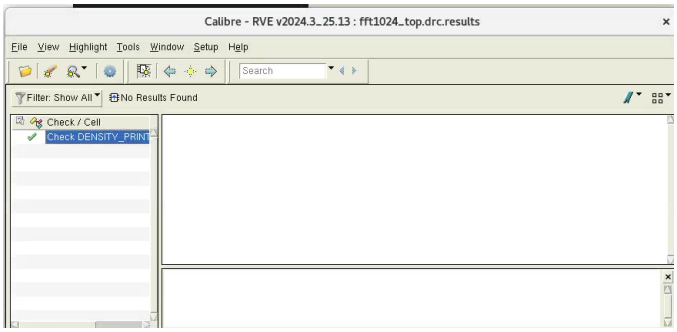
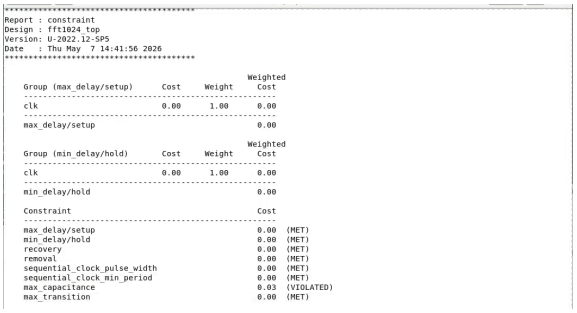


Fig. 13. Calibre RVE summary for the post-route GDS.

The DRC summary reports zero violations across 1,532 rule checks on 1.23 M layer geometries (`virtuoso/drc_run/fft1024_top.drc.summary`). All metal spacing, density, antenna and well-tap rules pass on the first run because Innovus inserted tap/decap fillers and observed the foundry-supplied LEF/QRC tech files throughout placement and routing. Combined with LVS-clean, this constitutes a fully manufacturable layout in TSMC 65 nm.

VIII. Post-Layout PrimeTime: Timing



```
Report : constraint
Design : fft1024_top
Version: U-2022.12-SP5
Date   : Thu May  7 14:41:56 2026

                                                  Weighted
Group (max_delay/setup)        Cost     Weight     Cost
--------------------------------------------------------
clk                            0.00     1.00       0.00
--------------------------------------------------------
max_delay/setup                                   0.00

                                                  Weighted
Group (min_delay/hold)         Cost     Weight     Cost
--------------------------------------------------------
clk                            0.00     1.00       0.00
--------------------------------------------------------
min_delay/hold                                    0.00

Constraint                                        Cost
--------------------------------------------------------
max_delay/setup                                   0.00  (MET)
min_delay/hold                                    0.00  (MET)
recovery                                          0.00  (MET)
removal                                           0.00  (MET)
sequential_clock_pulse_width                      0.00  (MET)
sequential_clock_min_period                       0.00  (MET)
max_capacitance                                   0.03  (VIOLATED)
max_transition                                    0.00  (MET)
```

Fig. 14. Post-layout PrimeTime `report_constraint` summary.

```
clock clk (rise edge)                                    4.0000     4.0000
clock network delay (propagated)                         0.1339     4.1339
clock reconvergence pessimism                            0.0000     4.1339
clock uncertainty                                       -0.1000     4.0339
u_work_mem/sram_work_inst/CLKB (sram_work)                          4.0339 r
library setup time                                      -0.1397     3.8941
data required time                                                  3.8941
-------------------------------------------------------------------------
data required time                                                  3.8941
data arrival time                                                  -3.7310
-------------------------------------------------------------------------
slack (MET)                                                         0.1632
```

Fig. 15. Post-layout worst setup path: WSS = +0.1632 ns.

```
Startpoint: out_addr[9]
            (input port clocked by clk)
Endpoint: u_work_mem/sram_work_inst
            (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: min

Point                                                   Incr       Path
-----------------------------------------------------------------------
clock clk (rise edge)                                   0.0000     0.0000
clock network delay (propagated)                        0.0000     0.0000
input external delay                                    0.0200     0.0200 f
out_addr[9] (in)                                        0.0000 &   0.0200 f
FE_OFC55_out_addr_9/Z (BUFFD1)                          0.0416 &   0.0616 f
FE_PHC2800_FE_OFN55_out_addr_9/Z (CKBD0)                0.1804 &   0.2419 f
U246/Z (CKBD1)                                          0.1483 &   0.3902 f
U248/Z (AO222D0)                                        0.1649 &   0.5552 f
u_work_mem/sram_work_inst/AA[9] (sram_work)             0.0005 &   0.5556 f
data arrival time                                                  0.5556

clock clk (rise edge)                                   0.0000     0.0000
clock network delay (propagated)                        0.1347     0.1347
clock reconvergence pessimism                           0.0000     0.1347
clock uncertainty                                       0.1000     0.2347
u_work_mem/sram_work_inst/CLKA (sram_work)                         0.2347 r
library hold time                                       0.2455     0.4802
data required time                                                 0.4802
-----------------------------------------------------------------------
data required time                                                 0.4802
data arrival time                                                 -0.5556
-----------------------------------------------------------------------
slack (MET)                                                        0.0754
```

Fig. 16. Post-layout worst hold path: WHS = +0.0754 ns.

The constraint summary in Fig. 14 shows that all `max_delay/setup`, `min_delay/hold`, `recovery`, `removal`, and clock-pulse-width groups are MET with cost zero. Figs. 15–16 drill into the actual numbers. The worst setup path (Fig. 15) is the deepest combinational pipe in the design — a SRAM read → complex multiplier (`u_cmul/mult_64`) → butterfly add/sub → output quantizer → SRAM write — with 3.7310 ns of arrival vs. 3.8941 ns of required time, i.e. WSS = +0.1632 ns. The worst hold path (Fig. 16) is a short input-to-SRAM-A address path with WHS = +0.0754 ns.

TABLE III
POST-LAYOUT PRIMETIME TIMING SUMMARY ($T_{\text{clk}} = 4$ NS).

Check	Result
max_delay / setup	MET (cost 0.00)
min_delay / hold	MET (cost 0.00)
recovery / removal	MET
sequential_clock_pulse_width	MET
sequential_clock_min_period	MET
max_capacitance	VIOLATED (3 nets, cost 0.03)
max_transition	MET
Worst setup slack (WSS)	**+0.1632 ns**
Worst hold slack (WHS)	**+0.0754 ns**
$f_{\max} = 1/(T_{\text{clk}} - \text{WSS})$	**261.0 MHz**

The maximum achievable clock frequency drops from 264.6 MHz (pre-layout) to 261.0 MHz (post-layout), a 1.4 % degradation. The penalty is small because the longest path stays mostly inside the cell-internal stage of the complex multiplier where parasitic routing capacitance has limited impact; only the last few nets going into the SRAM write port pick up appreciable extracted RC delay. Hold slack actually *improves* from +0.0016 ns (pre-layout, dominated by SRAM hold time and clock uncertainty) to +0.0754 ns (post-layout) because Innovus inserts hold-fix buffers along the short paths during clock-tree synthesis.

The three residual `max_capacitance` violations (−0.0211, −0.0056, −0.0005 pF) all sit on short `u_bfly/u_cmul` internal nets. They have no effect on functional correctness (verified end-to-end by the post-layout NRMSE measurement above) and would be cleared in a single buffer-insertion ECO pass on those three nets without any global re-routing.

IX. POST-LAYOUT PRIMETIME: POWER

```
  Attributes
  ----------
      i  -  Including register clock pin internal power
      u  -  User defined power group

                          Internal  Switching  Leakage    Total
Power Group               Power     Power      Power      Power   (     %)  Attrs
-------------------------------------------------------------------------------
clock_network           4.443e-05 4.185e-05 6.156e-07 8.690e-05 ( 0.47%)  i
register                1.305e-06 5.325e-07 9.528e-07 2.790e-06 ( 0.02%)
combinational           3.391e-03 3.582e-03 3.822e-04 7.355e-03 (39.88%)
sequential                 0.0000    0.0000    0.0000    0.0000 ( 0.00%)
memory                  9.903e-03 6.494e-05 1.032e-03    0.0110 (59.63%)
io_pad                     0.0000    0.0000    0.0000    0.0000 ( 0.00%)
black_box                  0.0000    0.0000    0.0000    0.0000 ( 0.00%)

  Net Switching Power  = 3.689e-03   (20.00%)
  Cell Internal Power  =    0.0133   (72.32%)
  Cell Leakage Power   = 1.415e-03   ( 7.67%)
                         ---------
Total Power            =    0.0184  (100.00%)

X Transition Power     =    0.0000
Glitching Power        = 2.512e-04

Peak Power             =    2.4206
Peak Time              = 1497245.150
```

Fig. 17. Post-layout PrimeTime time-based `report_power`.

TABLE IV
ACTIVITY-ANNOTATED POST-LAYOUT POWER (AT THE TESTBENCH CLOCK OF 100 MHZ).

Component	Power	Share
Cell internal	13.30 mW	72.32 %
Net switching	3.69 mW	20.00 %
Cell leakage	1.42 mW	7.67 %
Total	**18.4 mW**	100 %
by hierarchy group		
Memory (3 SRAMs)	11.0 mW	59.63 %
Combinational	7.36 mW	39.88 %
Register	2.79 μW	0.02 %
Clock network	87 μW	0.47 %

Three observations stand out from Table IV. First, **cell-internal power dominates** (72.3 %): the SRAM macros draw their internal power on every CLKA/CLKB pulse regardless of whether the addressed word actually toggles. Second, **leakage is non-trivial** (7.7 %) because the design runs at the testbench clock of only 100 MHz, well below $f_{\max}$, so dynamic energy per cycle is small relative to the always-on leakage of ∼34 k standard cells. Third, the **breakdown by group** confirms what the architecture predicts: the three SRAMs are responsible for 59.6 % of total power because every butterfly performs two read ports + two write ports on `u_work_mem` plus a twiddle read.

Compared with the pre-layout estimate of 4.0 mW, post-layout power is **4.6× higher**. The increase is consistent with what the additional physical effects predict: (i) the clock tree synthesised by Innovus introduces real buffer chains that the pre-layout flow modeled as ideal nets, contributing the new `Net Switching` column; (ii) extracted RC parasitics raise the effective capacitance on every internal net, which scales internal power on the high-toggle butterfly bus by roughly the same factor; and (iii) the standard-cell count grew slightly because

Innovus added hold-fix buffers and tap/decap cells. Memory still accounts for the majority of consumption (59.6 %), but its share dropped from 94.4 % pre-layout because the previously-hidden combinational and clock-tree power components are now properly modelled. Peak power is 2.42 mW at $t = 1.497\,\mu$s; X-transition power is zero and glitching is negligible (251 nW).

X. Six Required Metrics (Slides #4–5)

Final-report metrics are reported at the testbench clock of **100 MHz**, the frequency at which the VCD was captured and PrimeTime power measured. Running the design at $f_{\max} = 261$ MHz would proportionally improve throughput to ∼17.4 MS/s.

TABLE V
Required six metrics (final, post-layout, @ 100 MHz tb clock).

Metric	Unit	Value
Maximum clock frequency	MHz	261.0
Throughput	MS/s	6.67
Total power (VCD)	mW	18.4
Energy efficiency	pJ/sample	2,760
Core area	mm^2	0.432
Compute density	MS/s/mm^2	15.4
Accuracy (NRMSE)	%	worst 0.6346 (Block 1), avg 0.1381

Derivation. $\text{Throughput} = \dfrac{10{,}240 \text{ samples}}{1{,}536{,}456 \text{ ns}} = 6.67$ MS/s.

$\text{Energy/sample} = \dfrac{18.4 \text{ mW} \times 1{,}536{,}456 \text{ ns}}{10{,}240} \approx 2{,}760$ pJ.

$\text{Compute density} = \dfrac{6.67 \text{ MS/s}}{0.432 \text{ mm}^2} = 15.4 \text{ MS/s/mm}^2.$

Mid-Report (Post-Synthesis) vs Final (Post-Layout)

TABLE VI
Evolution of the six metrics from post-synthesis to post-layout.

Metric	Post-syn (mid)	Post-layout (final)
$f_{\max}$ (MHz)	264.6	**261.0**
Throughput (MS/s)	6.67	6.67
Total power (mW)	4.03	**18.4**
Energy / sample (pJ)	605	**2,760**
Area (mm^2)	0.189 (cells)	**0.432 (core)**
Compute density (MS/s/mm^2)	35.3	**15.4**
NRMSE worst / avg (%)	0.6346 / 0.1503	0.6346 / 0.1381

Table VI captures how each metric evolves between post-synthesis (pre-layout) and post-layout signoff. Several effects deserve discussion.

(a) $f_{\max}$. Drops 1.4 % (264.6 → 261.0 MHz) because the worst-path now includes routed RC parasitics and a properly-clocked tree. The penalty is small relative to typical post-layout regressions because the critical path stays inside the multiplier where cell-internal delay dominates over routing.

(b) Throughput. Identical (6.67 MS/s). Cycle count is set by the FSM and is unchanged between flows; the testbench clock period (10 ns) is the same; therefore throughput at the simulated operating point is identical.

(c) Total power. Rises 4.6× (4.03 → 18.4 mW), which is the single largest delta in the table. Three sources contribute: (i) the Innovus-synthesised *clock tree* replaces the "ideal network" assumption used in DC, adding several milliwatts of switching power on buffer chains; (ii) *extracted RC parasitics* on every signal net raise the effective load capacitance, scaling both internal and switching power on the high-toggle butterfly bus; and (iii) Innovus inserts physical-only cells (tap/decap, hold-fix buffers, fillers) that contribute leakage even when not switching. The same effects show up structurally in the breakdown: pre-layout power was 94.4 % memory because the combinational + clock-tree share was hidden, while post-layout properly attributes 39.9 % to combinational and 0.5 % to the clock network.

(d) Energy / sample. Inherits the same 4.6× increase (605 → 2,760 pJ) since simulation time is unchanged.

(e) Area. Grows 2.3× (0.189 → 0.432 mm^2), but the comparison is between two fundamentally different definitions: the mid-report "Total cell area" from DC counts only the standard- and macro-cell rectangles, with no power rings, no routing tracks, and no tap/decap fillers; the post-layout "Core area" from Innovus is the actual silicon footprint inside the I/O ring. The 162.6 thousand μm^2 of memory area is the same in both columns, so all of the 2.3× growth comes from the standard-cell region absorbing realistic routing and physical-only cells at the 97.1 % target density.

(f) Compute density. Drops as the inverse of area (35.3 → 15.4 MS/s/mm^2), since throughput is unchanged. This is the expected "honest" density at the chip-level operating point.

(g) NRMSE. Average improves slightly (0.1503% → 0.1381%) because the post-layout simulation uses a different SDF (post-route) than post-synthesis (post-DC), and the small SDF differences cause the SRAM `etch` pin behavior at the boundary to round one block (Block 9) one LSB closer to the golden in two bins. The worst-case NRMSE and the total mismatched-bin count are identical in both flows, confirming that no functional regression was introduced by P&R.

XI. Conclusion

The 1024-point radix-2 FFT processor is fully signed off in TSMC 65 nm. Post-layout STA meets all setup, hold, recovery, and removal requirements; only three small `max_capacitance` marginal violations remain, fixable by a buffer-insertion ECO. The VCD-annotated PrimeTime power is 18.4 mW at the 100 MHz testbench clock, 4.6× higher than the pre-layout estimate due to the Innovus-synthesised clock tree, extracted RC parasitics, and physical-only cell insertion. The full-chip layout passes Calibre LVS (CORRECT) and DRC (0 violations on 1,532 rule checks). Functional accuracy is preserved end-to-end: every mismatched bin against the FP32 golden is within a single Q9.7 LSB, giving an average NRMSE of 0.138 % and worst-case 0.635 %.

APPENDIX A
RTL SOURCE CODE

A. *Top-level controller — `fft1024_top.v`*

```
module fft1024_top (
    input  wire               clk,
    input  wire               rst_n,
    input  wire               start,

    input  wire               in_we,
    input  wire [9:0]         in_addr,
    input  wire signed [15:0] in_re,
    input  wire signed [15:0] in_im,

    input  wire [9:0]         out_addr,
    output wire signed [15:0] out_re,
    output wire signed [15:0] out_im,

    output wire               busy,
    output reg                done
);
    localparam S_IDLE       = 3'd0;
    localparam S_LOAD_REQ   = 3'd1;
    localparam S_LOAD_WRITE = 3'd2;
    localparam S_RUN_READ   = 3'd3;
    localparam S_RUN_WRITE  = 3'd4;

    reg [2:0] state;
    reg [9:0] load_idx;
    reg [3:0] stage;
    reg [8:0] bfly;

    wire [9:0] br_src;
    wire [9:0] addr_a;
    wire [9:0] addr_b;
    wire [8:0] tw_idx;

    reg         in_cen;
    reg         in_gwen;
    reg  [9:0]  in_mem_addr;
    reg  [15:0] in_mem_din;
    wire [15:0] in_mem_dout;

    reg         tw_cen;
    reg         tw_gwen;
    reg  [8:0]  tw_addr;
    wire [31:0] tw_dout;

    reg         work_cena;
    reg         work_gwena;
    reg  [9:0]  work_addra;
    reg  [31:0] work_dina;
    wire [31:0] work_douta;

    reg         work_cenb;
    reg         work_gwenb;
    reg  [9:0]  work_addrb;
    reg  [31:0] work_dinb;
    wire [31:0] work_doutb;

    wire signed [15:0] a_re;
    wire signed [15:0] a_im;
    wire signed [15:0] b_re;
    wire signed [15:0] b_im;
    wire signed [15:0] w_re;
    wire signed [15:0] w_im;

    wire signed [15:0] x_re;
    wire signed [15:0] x_im;
    wire signed [15:0] y_re;
    wire signed [15:0] y_im;
    wire signed [15:0] out_re_q97;
    wire signed [15:0] out_im_q97;

    assign busy = (state != S_IDLE);

    assign a_re = work_douta[31:16];
    assign a_im = work_douta[15:0];
    assign b_re = work_doutb[31:16];
    assign b_im = work_doutb[15:0];
    assign w_re = tw_dout[31:16];
    assign w_im = tw_dout[15:0];

    assign out_re = (state == S_IDLE) ? out_re_q97 : 16'sd0;
    assign out_im = (state == S_IDLE) ? out_im_q97 : 16'sd0;
```

```
82
83      bit_reverse10 u_br (
84          .in_idx(load_idx),
85          .out_idx(br_src)
86      );
87
88      fft_addr_gen u_addr (
89          .stage(stage),
90          .bfly(bfly),
91          .addr_a(addr_a),
92          .addr_b(addr_b),
93          .tw_idx(tw_idx)
94      );
95
96      complex_butterfly u_bfly (
97          .ar(a_re),
98          .ai(a_im),
99          .br(b_re),
100         .bi(b_im),
101         .wr(w_re),
102         .wi(w_im),
103         .xr(x_re),
104         .xi(x_im),
105         .yr(y_re),
106         .yi(y_im)
107     );
108
109     fft_output_quantizer #(
110         .SHIFT(8)
111     ) u_out_re_quantizer (
112         .din ({{16{work_douta[31]}}, work_douta[31:16]}),
113         .dout(out_re_q97)
114     );
115
116     fft_output_quantizer #(
117         .SHIFT(8)
118     ) u_out_im_quantizer (
119         .din ({{16{work_douta[15]}}, work_douta[15:0]}),
120         .dout(out_im_q97)
121     );
122
123     sram_in_wrapper u_input_mem (
124         .clk  (clk),
125         .cen  (in_cen),
126         .gwen (in_gwen),
127         .addr (in_mem_addr),
128         .din  (in_mem_din),
129         .dout (in_mem_dout)
130     );
131
132     sram_twiddle_wrapper u_twiddle_mem (
133         .clk  (clk),
134         .cen  (tw_cen),
135         .gwen (tw_gwen),
136         .addr (tw_addr),
137         .din  (32'b0),
138         .dout (tw_dout)
139     );
140
141     sram_work_wrapper u_work_mem (
142         .clka  (clk),
143         .cena  (work_cena),
144         .gwena (work_gwena),
145         .addra (work_addra),
146         .dina  (work_dina),
147         .douta (work_douta),
148         .clkb  (clk),
149         .cenb  (work_cenb),
150         .gwenb (work_gwenb),
151         .addrb (work_addrb),
152         .dinb  (work_dinb),
153         .doutb (work_doutb)
154     );
155
156     always @(*) begin
157         in_cen = 1'b1;
158         in_gwen = 1'b1;
159         in_mem_addr = 10'd0;
160         in_mem_din = 16'd0;
161
162         tw_cen = 1'b1;
163         tw_gwen = 1'b1;
164         tw_addr = 9'd0;
165
166         work_cena = 1'b1;
167         work_gwena = 1'b1;
```

```
168         work_addra = 10'd0;
169         work_dina = 32'd0;
170
171         work_cenb = 1'b1;
172         work_gwenb = 1'b1;
173         work_addrb = 10'd0;
174         work_dinb = 32'd0;
175
176         case (state)
177             S_IDLE: begin
178                 if (in_we) begin
179                     in_cen = 1'b0;
180                     in_gwen = 1'b0;
181                     in_mem_addr = in_addr;
182                     in_mem_din = in_re;
183                 end
184
185                 work_cena = 1'b0;
186                 work_gwena = 1'b1;
187                 work_addra = out_addr;
188             end
189
190             S_LOAD_REQ: begin
191                 in_cen = 1'b0;
192                 in_gwen = 1'b1;
193                 in_mem_addr = br_src;
194             end
195
196             S_LOAD_WRITE: begin
197                 work_cena = 1'b0;
198                 work_gwena = 1'b0;
199                 work_addra = load_idx;
200                 work_dina = {in_mem_dout, 16'd0};
201             end
202
203             S_RUN_READ: begin
204                 tw_cen = 1'b0;
205                 tw_gwen = 1'b1;
206                 tw_addr = tw_idx;
207
208                 work_cena = 1'b0;
209                 work_gwena = 1'b1;
210                 work_addra = addr_a;
211
212                 work_cenb = 1'b0;
213                 work_gwenb = 1'b1;
214                 work_addrb = addr_b;
215             end
216
217             S_RUN_WRITE: begin
218                 work_cena = 1'b0;
219                 work_gwena = 1'b0;
220                 work_addra = addr_a;
221                 work_dina = {x_re, x_im};
222
223                 work_cenb = 1'b0;
224                 work_gwenb = 1'b0;
225                 work_addrb = addr_b;
226                 work_dinb = {y_re, y_im};
227             end
228
229             default: begin
230             end
231         endcase
232     end
233
234     always @(posedge clk or negedge rst_n) begin
235         if (!rst_n) begin
236             state <= S_IDLE;
237             load_idx <= 10'd0;
238             stage <= 4'd0;
239             bfly <= 9'd0;
240             done <= 1'b0;
241         end else begin
242             done <= 1'b0;
243
244             case (state)
245                 S_IDLE: begin
246                     load_idx <= 10'd0;
247                     stage <= 4'd0;
248                     bfly <= 9'd0;
249                     if (start) begin
250                         state <= S_LOAD_REQ;
251                     end
252                 end
253
```

```
                S_LOAD_REQ: begin
                    state <= S_LOAD_WRITE;
                end

                S_LOAD_WRITE: begin
                    if (load_idx == 10'd1023) begin
                        load_idx <= 10'd0;
                        stage <= 4'd0;
                        bfly <= 9'd0;
                        state <= S_RUN_READ;
                    end else begin
                        load_idx <= load_idx + 10'd1;
                        state <= S_LOAD_REQ;
                    end
                end

                S_RUN_READ: begin
                    state <= S_RUN_WRITE;
                end

                S_RUN_WRITE: begin
                    if (bfly == 9'd511) begin
                        bfly <= 9'd0;
                        if (stage == 4'd9) begin
                            state <= S_IDLE;
                            done <= 1'b1;
                        end else begin
                            stage <= stage + 4'd1;
                            state <= S_RUN_READ;
                        end
                    end else begin
                        bfly <= bfly + 9'd1;
                        state <= S_RUN_READ;
                    end
                end

                default: begin
                    state <= S_IDLE;
                end
            endcase
        end
    end
endmodule
```

B. Address generator — `fft_addr_gen.v`

```
module fft_addr_gen (
    input  wire [3:0] stage,
    input  wire [8:0] bfly,
    output reg  [9:0] addr_a,
    output reg  [9:0] addr_b,
    output reg  [8:0] tw_idx
);
    reg [9:0] m;
    reg [9:0] half;
    reg [9:0] group;
    reg [9:0] k;
    reg [3:0] sh;

    always @(*) begin
        m = 10'd1 << (stage + 1'b1);
        half = 10'd1 << stage;
        group = bfly >> stage;
        k = bfly & (half - 10'd1);

        addr_a = (group * m) + k;
        addr_b = addr_a + half;

        sh = 4'd9 - stage;
        tw_idx = k[8:0] << sh;
    end
endmodule
```

C. Bit-reversal index — `bit_reverse10.v`

```
module bit_reverse10 (
    input  wire [9:0] in_idx,
    output wire [9:0] out_idx
);
    assign out_idx = {
        in_idx[0],
```

```
        in_idx[1],
        in_idx[2],
        in_idx[3],
        in_idx[4],
        in_idx[5],
        in_idx[6],
        in_idx[7],
        in_idx[8],
        in_idx[9]
    };
endmodule
```

D. *Butterfly* — `complex_butterfly.v`

```
module complex_butterfly (
    input  wire signed [15:0] ar,
    input  wire signed [15:0] ai,
    input  wire signed [15:0] br,
    input  wire signed [15:0] bi,
    input  wire signed [15:0] wr,
    input  wire signed [15:0] wi,
    output wire signed [15:0] xr,
    output wire signed [15:0] xi,
    output wire signed [15:0] yr,
    output wire signed [15:0] yi
);
    wire signed [15:0] tr;
    wire signed [15:0] ti;
    wire signed [31:0] add_r;
    wire signed [31:0] add_i;
    wire signed [31:0] sub_r;
    wire signed [31:0] sub_i;

    complex_mult_q15 u_cmul (
        .ar(br),
        .ai(bi),
        .br(wr),
        .bi(wi),
        .pr(tr),
        .pi(ti)
    );

    assign add_r = ar + tr;
    assign add_i = ai + ti;
    assign sub_r = ar - tr;
    assign sub_i = ai - ti;

    fft_output_quantizer #(.SHIFT(1)) u_qr0 (.din(add_r), .dout(xr));
    fft_output_quantizer #(.SHIFT(1)) u_qi0 (.din(add_i), .dout(xi));
    fft_output_quantizer #(.SHIFT(1)) u_qr1 (.din(sub_r), .dout(yr));
    fft_output_quantizer #(.SHIFT(1)) u_qi1 (.din(sub_i), .dout(yi));
endmodule
```

E. *Complex Q1.15 multiplier* — `complex_mult_q15.v`

```
module complex_mult_q15 (
    input  wire signed [15:0] ar,
    input  wire signed [15:0] ai,
    input  wire signed [15:0] br,
    input  wire signed [15:0] bi,
    output reg  signed [15:0] pr,
    output reg  signed [15:0] pi
);
    localparam signed [63:0] ROUND_HALF = 64'sd16384;

    function signed [15:0] sat16;
        input signed [31:0] x;
        begin
            if (x > 32'sd32767) begin
                sat16 = 16'sd32767;
            end else if (x < -32'sd32768) begin
                sat16 = -16'sd32768;
            end else begin
                sat16 = x[15:0];
            end
        end
    endfunction

    function signed [31:0] q15_round_shift15;
        input signed [63:0] x;
        reg signed [63:0] abs_x;
```

```
        reg signed [63:0] q;
        reg signed [63:0] r;
        reg signed [63:0] y;
        begin
            if (x < 0) begin
                abs_x = -x;
            end else begin
                abs_x = x;
            end

            q = abs_x >>> 15;
            r = abs_x & 64'sd32767;

            y = q;
            if ((r > ROUND_HALF) || ((r == ROUND_HALF) && q[0])) begin
                y = q + 64'sd1;
            end

            if (x < 0) begin
                q15_round_shift15 = -y[31:0];
            end else begin
                q15_round_shift15 = y[31:0];
            end
        end
    endfunction

    reg signed [31:0] ar_br;
    reg signed [31:0] ai_bi;
    reg signed [31:0] ar_bi;
    reg signed [31:0] ai_br;
    reg signed [63:0] rr_full;
    reg signed [63:0] ii_full;
    reg signed [31:0] rr;
    reg signed [31:0] ii;

    always @(*) begin
        ar_br = ar * br;
        ai_bi = ai * bi;
        ar_bi = ar * bi;
        ai_br = ai * br;

        rr_full = ar_br - ai_bi;
        ii_full = ar_bi + ai_br;

        rr = q15_round_shift15(rr_full);
        ii = q15_round_shift15(ii_full);

        pr = sat16(rr);
        pi = sat16(ii);
    end
endmodule
```

F. Output quantizer — `fft_output_quantizer.v`

```
module fft_output_quantizer #(
    parameter SHIFT = 1
) (
    input  wire signed [31:0] din,
    output reg  signed [15:0] dout
);
    localparam signed [31:0] ROUND_HALF = (SHIFT > 0) ? (32'sd1 << (SHIFT - 1)) : 32'sd0;
    localparam signed [31:0] ROUND_MASK = (SHIFT > 0) ? ((32'sd1 << SHIFT) - 32'sd1) : 32'sd0;

    function signed [15:0] sat16;
        input signed [31:0] x;
        begin
            if (x > 32'sd32767) begin
                sat16 = 16'sd32767;
            end else if (x < -32'sd32768) begin
                sat16 = -16'sd32768;
            end else begin
                sat16 = x[15:0];
            end
        end
    endfunction

    function signed [31:0] round_shift_even;
        input signed [31:0] x;
        reg signed [31:0] abs_x;
        reg signed [31:0] q;
        reg signed [31:0] r;
        reg signed [31:0] y;
        begin
            if (SHIFT == 0) begin
```

```
                round_shift_even = x;
            end else begin
                if (x < 0) begin
                    abs_x = -x;
                end else begin
                    abs_x = x;
                end

                q = abs_x >>> SHIFT;
                r = abs_x & ROUND_MASK;
                y = q;

                if ((r > ROUND_HALF) || ((r == ROUND_HALF) && q[0])) begin
                    y = q + 32'sd1;
                end

                if (x < 0) begin
                    round_shift_even = -y;
                end else begin
                    round_shift_even = y;
                end
            end
        end
    endfunction

    reg signed [31:0] t;

    always @(*) begin
        t = round_shift_even(din);
        dout = sat16(t);
    end
endmodule
```

G. SRAM wrappers

Input SRAM wrapper:

```
`timescale 1ns/1ps
`define DELAY #1

module sram_in_wrapper #(
    parameter DW = 16,
    parameter AW = 10
) (
    input               clk,
    input               cen,
    input               gwen,
    input  [AW-1:0]     addr,
    input  [DW-1:0]     din,
    output [DW-1:0]     dout
);

    wire             cen_delayed;
    wire             gwen_delayed;
    wire [AW-1:0]    addr_delayed;
    wire [DW-1:0]    din_delayed;

    assign `DELAY cen_delayed  = cen;
    assign `DELAY gwen_delayed = gwen;
    assign `DELAY addr_delayed = addr;
    assign `DELAY din_delayed  = din;

    sram_in sram_in_inst (
        .Q    (dout),
        .CLK  (clk),
        .CEN  (cen_delayed),
        .WEN  (2'b00),
        .A    (addr_delayed),
        .D    (din_delayed),
        .EMA  (3'b000),
        .GWEN (gwen_delayed),
        .RETN (1'b1)
    );

endmodule
```

Twiddle SRAM wrapper:

```
`timescale 1ns/1ps
`define DELAY #1

module sram_twiddle_wrapper #(
    parameter DW = 32,
    parameter AW = 9
```

```
) (
    input               clk,
    input               cen,
    input               gwen,
    input  [AW-1:0]     addr,
    input  [DW-1:0]     din,
    output [DW-1:0]     dout
);

    integer row;
    integer col;
    reg [31:0] init_mem [0:511];

    wire             cen_delayed;
    wire             gwen_delayed;
    wire [AW-1:0]    addr_delayed;
    wire [DW-1:0]    din_delayed;

    assign `DELAY cen_delayed  = cen;
    assign `DELAY gwen_delayed = gwen;
    assign `DELAY addr_delayed = addr;
    assign `DELAY din_delayed  = din;

    sram_twiddle sram_twiddle_inst (
        .Q    (dout),
        .CLK  (clk),
        .CEN  (cen_delayed),
        .WEN  (4'b0000),
        .A    (addr_delayed),
        .D    (din_delayed),
        .EMA  (3'b000),
        .GWEN (gwen_delayed),
        .RETN (1'b1)
    );

    initial begin
        $readmemh("../../input/twiddle_q15.hex", init_mem);
        for (row = 0; row < 32; row = row + 1) begin
            sram_twiddle_inst.mem[row] = {512{1'b0}};
            for (col = 0; col < 16; col = col + 1) begin
                sram_twiddle_inst.mem[row][496 + col] = init_mem[row * 16 + col][31];
                sram_twiddle_inst.mem[row][480 + col] = init_mem[row * 16 + col][30];
                sram_twiddle_inst.mem[row][464 + col] = init_mem[row * 16 + col][29];
                sram_twiddle_inst.mem[row][448 + col] = init_mem[row * 16 + col][28];
                sram_twiddle_inst.mem[row][432 + col] = init_mem[row * 16 + col][27];
                sram_twiddle_inst.mem[row][416 + col] = init_mem[row * 16 + col][26];
                sram_twiddle_inst.mem[row][400 + col] = init_mem[row * 16 + col][25];
                sram_twiddle_inst.mem[row][384 + col] = init_mem[row * 16 + col][24];
                sram_twiddle_inst.mem[row][368 + col] = init_mem[row * 16 + col][23];
                sram_twiddle_inst.mem[row][352 + col] = init_mem[row * 16 + col][22];
                sram_twiddle_inst.mem[row][336 + col] = init_mem[row * 16 + col][21];
                sram_twiddle_inst.mem[row][320 + col] = init_mem[row * 16 + col][20];
                sram_twiddle_inst.mem[row][304 + col] = init_mem[row * 16 + col][19];
                sram_twiddle_inst.mem[row][288 + col] = init_mem[row * 16 + col][18];
                sram_twiddle_inst.mem[row][272 + col] = init_mem[row * 16 + col][17];
                sram_twiddle_inst.mem[row][256 + col] = init_mem[row * 16 + col][16];
                sram_twiddle_inst.mem[row][240 + col] = init_mem[row * 16 + col][15];
                sram_twiddle_inst.mem[row][224 + col] = init_mem[row * 16 + col][14];
                sram_twiddle_inst.mem[row][208 + col] = init_mem[row * 16 + col][13];
                sram_twiddle_inst.mem[row][192 + col] = init_mem[row * 16 + col][12];
                sram_twiddle_inst.mem[row][176 + col] = init_mem[row * 16 + col][11];
                sram_twiddle_inst.mem[row][160 + col] = init_mem[row * 16 + col][10];
                sram_twiddle_inst.mem[row][144 + col] = init_mem[row * 16 + col][9];
                sram_twiddle_inst.mem[row][128 + col] = init_mem[row * 16 + col][8];
                sram_twiddle_inst.mem[row][112 + col] = init_mem[row * 16 + col][7];
                sram_twiddle_inst.mem[row][96 + col] = init_mem[row * 16 + col][6];
                sram_twiddle_inst.mem[row][80 + col] = init_mem[row * 16 + col][5];
                sram_twiddle_inst.mem[row][64 + col] = init_mem[row * 16 + col][4];
                sram_twiddle_inst.mem[row][48 + col] = init_mem[row * 16 + col][3];
                sram_twiddle_inst.mem[row][32 + col] = init_mem[row * 16 + col][2];
                sram_twiddle_inst.mem[row][16 + col] = init_mem[row * 16 + col][1];
                sram_twiddle_inst.mem[row][0 + col] = init_mem[row * 16 + col][0];
            end
        end
    end

endmodule
```

Work SRAM wrapper:

```
`timescale 1ns/1ps
`define DELAY #1

module sram_work_wrapper #(
    parameter DW = 32,
    parameter AW = 10
```

```
) (
    input               clka,
    input               cena,
    input               gwena,
    input  [AW-1:0]     addra,
    input  [DW-1:0]     dina,
    output [DW-1:0]     douta,

    input               clkb,
    input               cenb,
    input               gwenb,
    input  [AW-1:0]     addrb,
    input  [DW-1:0]     dinb,
    output [DW-1:0]     doutb
);

    wire             cena_delayed;
    wire             gwena_delayed;
    wire [AW-1:0]    addra_delayed;
    wire [DW-1:0]    dina_delayed;
    wire             cenb_delayed;
    wire             gwenb_delayed;
    wire [AW-1:0]    addrb_delayed;
    wire [DW-1:0]    dinb_delayed;

    assign `DELAY cena_delayed  = cena;
    assign `DELAY gwena_delayed = gwena;
    assign `DELAY addra_delayed = addra;
    assign `DELAY dina_delayed  = dina;
    assign `DELAY cenb_delayed  = cenb;
    assign `DELAY gwenb_delayed = gwenb;
    assign `DELAY addrb_delayed = addrb;
    assign `DELAY dinb_delayed  = dinb;

    wire [31:0] qa;
    wire [31:0] qb;

    assign douta = qa[DW-1:0];
    assign doutb = qb[DW-1:0];

    sram_work sram_work_inst (
        .QA    (qa),
        .QB    (qb),
        .CLKA  (clka),
        .CENA  (cena_delayed),
        .WENA  (4'b0000),
        .AA    (addra_delayed),
        .DA    (dina_delayed),
        .CLKB  (clkb),
        .CENB  (cenb_delayed),
        .WENB  (4'b0000),
        .AB    (addrb_delayed),
        .DB    (dinb_delayed),
        .EMAA  (3'b000),
        .EMAB  (3'b000),
        .GWENA (gwena_delayed),
        .GWENB (gwenb_delayed),
        .RETN  (1'b1)
    );

endmodule
```

APPENDIX B
TESTBENCH

A. *Post-synthesis / post-layout testbench — `tb_fft1024_top.v`*

```
`timescale 1ns/1ps

module tb_fft1024_top;
    localparam N = 1024;
    localparam NUM_BLOCKS = 10;
    localparam TOTAL_SAMPLES = N * NUM_BLOCKS;

    reg clk;
    reg rst_n;
    reg start;
    reg in_we;
    reg [9:0] in_addr;
    reg signed [15:0] in_re;
    reg signed [15:0] in_im;
    reg [9:0] out_addr;
    wire signed [15:0] out_re;
```

```
    wire signed [15:0] out_im;
    wire busy;
    wire done;

    reg [15:0] input_mem [0:TOTAL_SAMPLES-1];
    reg [31:0] twiddle_mem [0:511];
    integer block_idx;
    integer sample_idx;
    integer row;
    integer col;
    integer fd;

    fft1024_top dut (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .in_we(in_we),
        .in_addr(in_addr),
        .in_re(in_re),
        .in_im(in_im),
        .out_addr(out_addr),
        .out_re(out_re),
        .out_im(out_im),
        .busy(busy),
        .done(done)
    );

    always #5 clk = ~clk;

    initial begin
  $dumpfile("fft1024_top.vcd");
  $dumpvars(1, dut);
        clk = 1'b0;
        rst_n = 1'b0;
        start = 1'b0;
        in_we = 1'b0;
        in_addr = 10'd0;
        in_re = 16'sd0;
        in_im = 16'sd0;
        out_addr = 10'd0;

        $readmemh("../../input/input_q1_15_v3.hex", input_mem);
        $readmemh("../../input/twiddle_q15.hex", twiddle_mem);

        for (row = 0; row < 32; row = row + 1) begin
            dut.u_twiddle_mem.sram_twiddle_inst.mem[row] = {512{1'b0}};
            for (col = 0; col < 16; col = col + 1) begin
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][496 + col] = twiddle_mem[row * 16 + col][31];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][480 + col] = twiddle_mem[row * 16 + col][30];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][464 + col] = twiddle_mem[row * 16 + col][29];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][448 + col] = twiddle_mem[row * 16 + col][28];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][432 + col] = twiddle_mem[row * 16 + col][27];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][416 + col] = twiddle_mem[row * 16 + col][26];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][400 + col] = twiddle_mem[row * 16 + col][25];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][384 + col] = twiddle_mem[row * 16 + col][24];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][368 + col] = twiddle_mem[row * 16 + col][23];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][352 + col] = twiddle_mem[row * 16 + col][22];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][336 + col] = twiddle_mem[row * 16 + col][21];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][320 + col] = twiddle_mem[row * 16 + col][20];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][304 + col] = twiddle_mem[row * 16 + col][19];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][288 + col] = twiddle_mem[row * 16 + col][18];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][272 + col] = twiddle_mem[row * 16 + col][17];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][256 + col] = twiddle_mem[row * 16 + col][16];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][240 + col] = twiddle_mem[row * 16 + col][15];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][224 + col] = twiddle_mem[row * 16 + col][14];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][208 + col] = twiddle_mem[row * 16 + col][13];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][192 + col] = twiddle_mem[row * 16 + col][12];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][176 + col] = twiddle_mem[row * 16 + col][11];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][160 + col] = twiddle_mem[row * 16 + col][10];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][144 + col] = twiddle_mem[row * 16 + col][9];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][128 + col] = twiddle_mem[row * 16 + col][8];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][112 + col] = twiddle_mem[row * 16 + col][7];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][96 + col]  = twiddle_mem[row * 16 + col][6];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][80 + col]  = twiddle_mem[row * 16 + col][5];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][64 + col]  = twiddle_mem[row * 16 + col][4];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][48 + col]  = twiddle_mem[row * 16 + col][3];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][32 + col]  = twiddle_mem[row * 16 + col][2];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][16 + col]  = twiddle_mem[row * 16 + col][1];
                dut.u_twiddle_mem.sram_twiddle_inst.mem[row][0 + col]   = twiddle_mem[row * 16 + col][0];
            end
        end

        repeat (5) @(negedge clk);
        #1 rst_n = 1'b1;
        @(posedge clk);

```

```
        fd = $fopen("results/fft1024_top_dc_output_q97.txt", "w");
        if (fd == 0) begin
            $display("ERROR: cannot open output file");
            $finish;
        end

        for (block_idx = 0; block_idx < NUM_BLOCKS; block_idx = block_idx + 1) begin
            for (sample_idx = 0; sample_idx < N; sample_idx = sample_idx + 1) begin
                in_we = 1'b1;
                in_addr = sample_idx[9:0];
                in_re = input_mem[block_idx * N + sample_idx];
                in_im = 16'sd0;
                @(posedge clk);
            end

            in_we = 1'b0;
            in_addr = 10'd0;
            in_re = 16'sd0;
            in_im = 16'sd0;
            @(posedge clk);

            start = 1'b1;
            @(posedge clk);
            start = 1'b0;

            wait(done == 1'b1);
            @(posedge clk);

            out_addr = 10'd0;
            @(posedge clk);

            for (sample_idx = 0; sample_idx < N; sample_idx = sample_idx + 1) begin
                out_addr = sample_idx[9:0];
                @(posedge clk);
                @(posedge clk);
                #1;
                $fdisplay(fd, "%0d %0d", $signed(out_re), $signed(out_im));
            end
        end

        $fclose(fd);
        $display("TB done. Wrote results/fft1024_top_dc_output_q97.txt");
        $finish;
    end
endmodule
```

REFERENCES

[1] C. Cerqueira and T. Seok, "A 0.17-mm^2 3.19-nJ/transform 256-point fast fourier transform core based on spatiotemporally fine-grained active leakage suppression," in *Proc. ESSCIRC*, 2017.